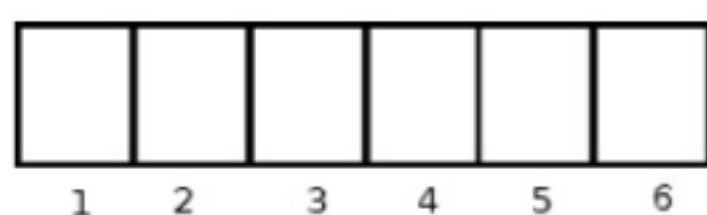


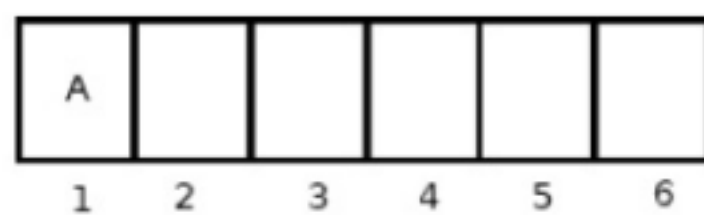
Poo: mantener la interfaz

- Siempre que cambiemos la implementación de una clase debemos buscar mantener la interfaz (cabecera o firma de los métodos públicos) para no afectar a los usuarios.
- El ejemplo de uso de una pila que hicimos utilizando la implementación estática, corre de la misma manera que con la implementación dinámica porque no hicimos cambios en la interfaz.

Cola: Implementación estática

pri = 0
ulti = 0

(a) Cola vacía



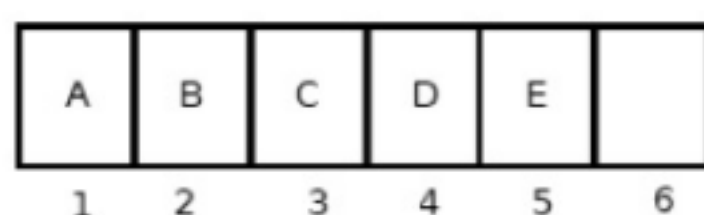
pri
ulti

(b) Cola con un elemento



pri ulti

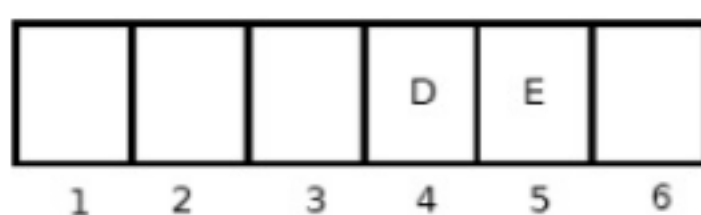
(c) Cola con dos elementos



pri

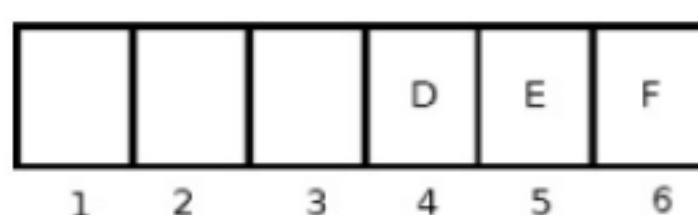
ulti

(d) Cola con 5 elementos



pri ulti

(e) Se eliminan tres elementos

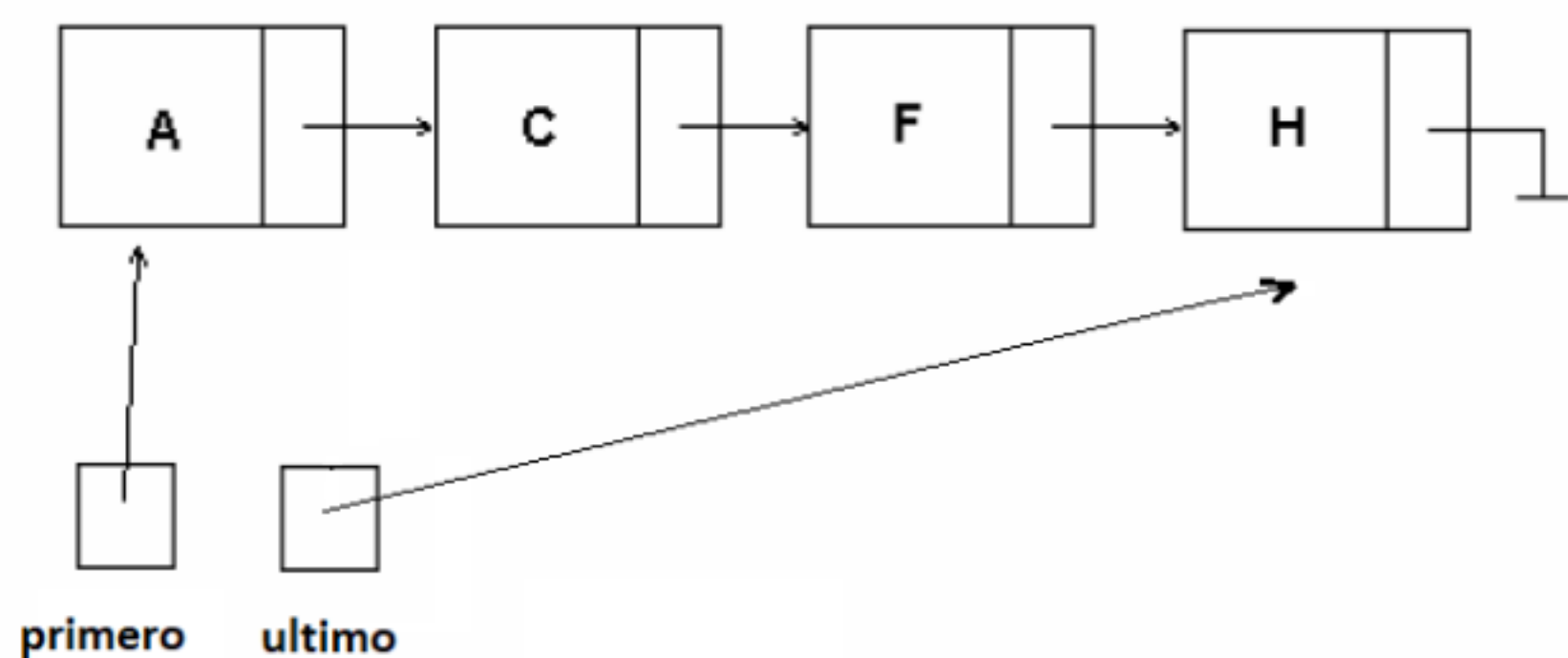


pri

ulti

(f) Cola que parece llena

Cola: Implementación dinámica



Lista: operaciones

Alta: agrega un elemento en la lista. Puede ser:

- $alta(x)$
- $alta(x, posición)$

Baja

- $baja(x)$
- $baja(posición)$

Consultar

- $Consultar(x.clave)$
- $Consultar(posición)$

Lista: operación de alta

$alta(x)$ la lista podría estar ordenada por algún campo clave, entonces se compara $x.clave$ con los campos clave de los elementos de la lista, insertándose x en el lugar correspondiente.

alta(x, Posicion)

Posicion es el lugar en donde debe insertarse el elemento x .

Por ejemplo, en una lista $l_1, l_2, l_3, \dots, l_n$

$\text{alta}(x, 1)$; // genera $x, l_1, l_2, l_3, \dots, l_n$

Si posicion excede en más de 1 al tamaño de la lista hay que decidir qué hacer.

Lista: operaciones de bajabaja(x)

Busca el elemento x en la lista y lo borra.

También podría ser sólo una clave, por lo que buscaría x . clave. Si no lo encuentra, no hace nada.

Hay que decidir qué hacer con los elementos repetidos, si los hubiera.

baja(Posicion)

Borra el elemento de la lista que se encuentra en la posición Posicion. Las decisiones en cuanto a si posición es mayor que n (la cantidad de elementos de la lista) son las mismas que en el apartado anterior, es decir, no hacer nada, borrar el último elemento o lanzar una excepción.

Lista: operación de consulta

Consultar (x. clave)

Se tiene una parte del elemento, solo la clave, y se desea recuperarlo en su totalidad.

Consultar (posición)

Se devuelve el elemento que está en posición.

Nota: En ambas opciones, el resultado debe devolverse.

Decidir: ¿Qué devolvería el método si la clave no se encuentra?

Lista: otras operaciones

Pueden haber más operaciones que las tres mencionadas, pero esas son las básicas a partir de las cuales se pueden obtener otras.

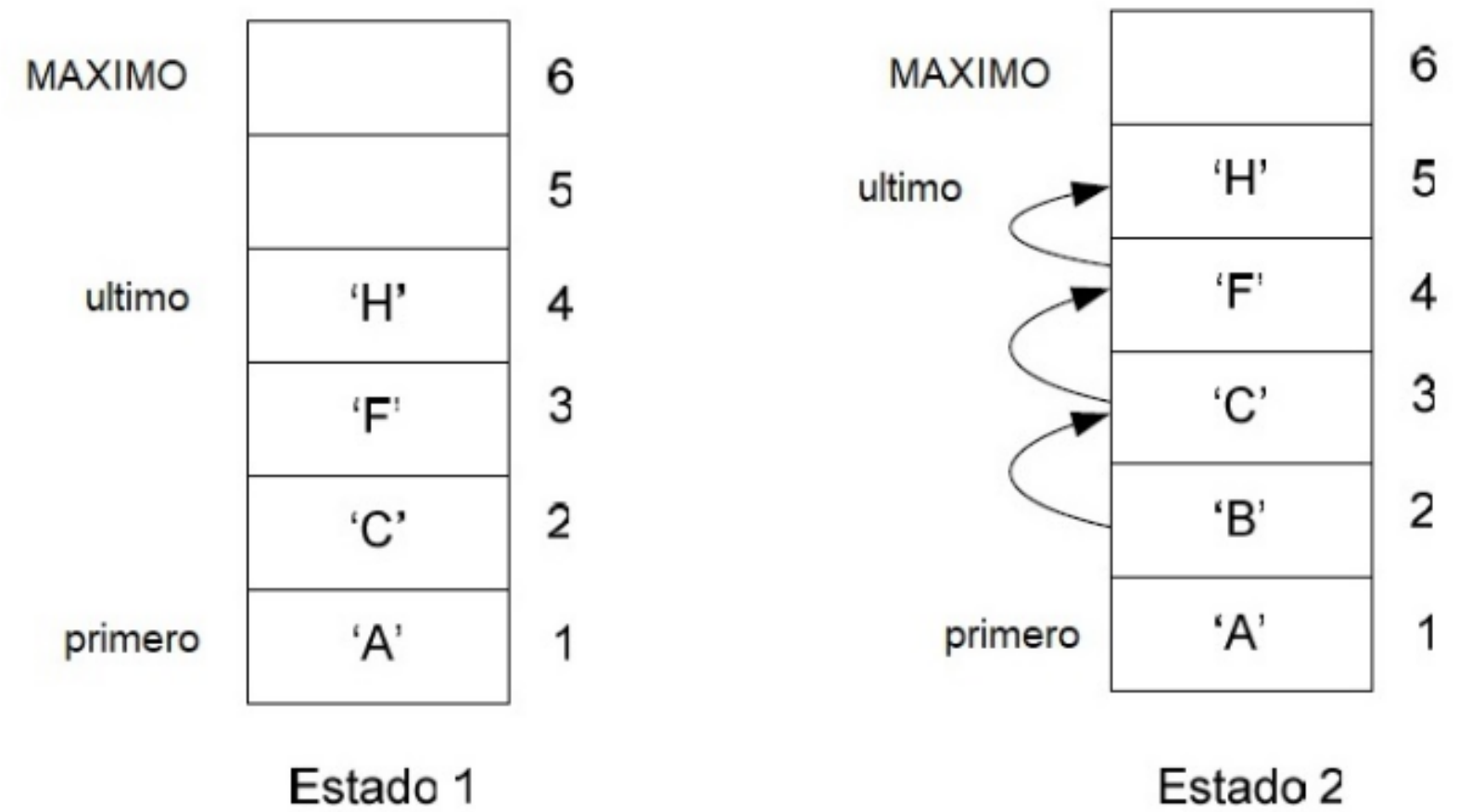
Por ejemplo, se podría desear tener un borrado completo de la lista, pero esta operación se podría realizar llamando al borrado del primer elemento hasta que la lista quede vacía.

otras operaciones

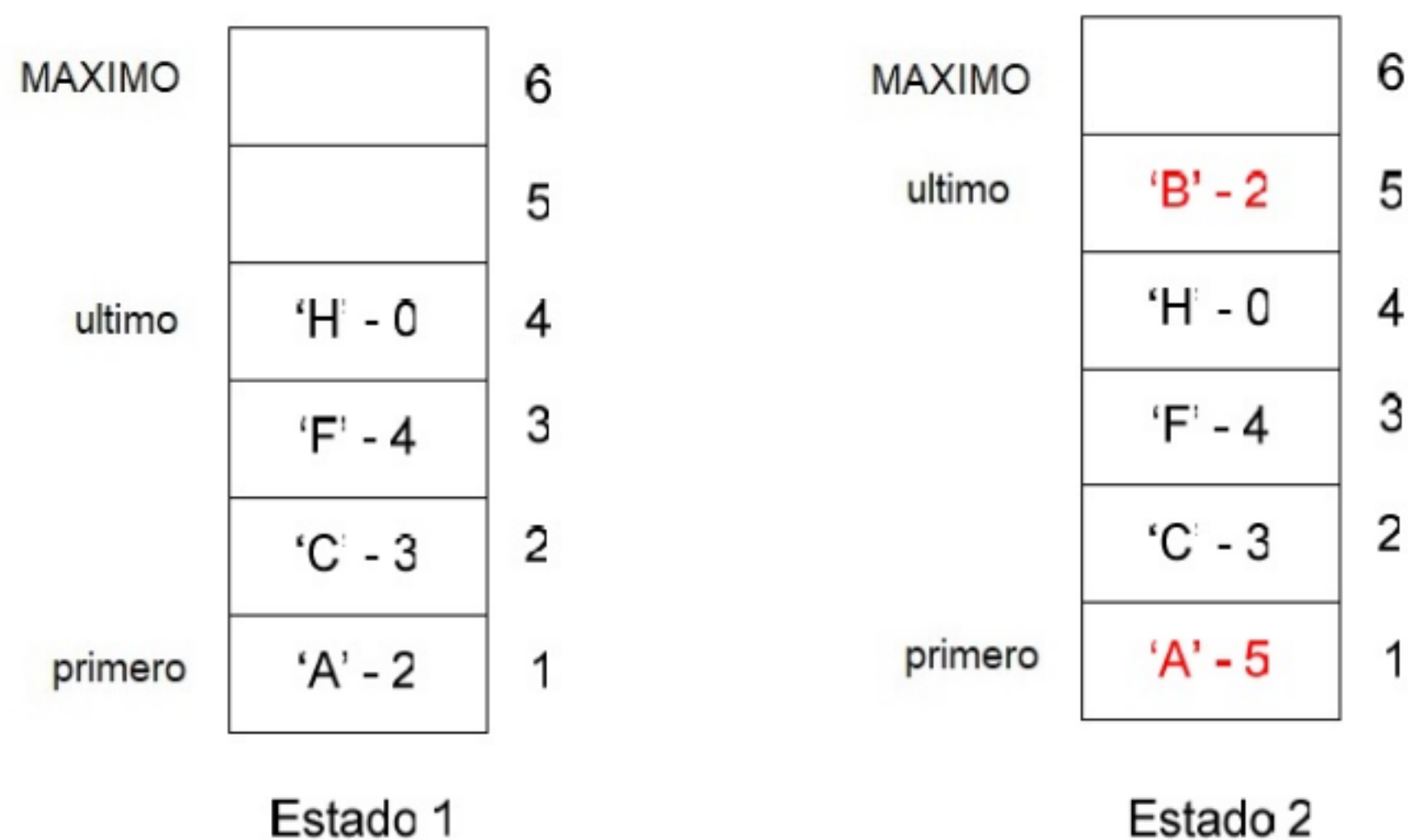
- vacia
- obtenerTamano
- copiar
- etc

Lista: implementaciones estáticas

En la implementación estática se agrega el problema de los corrimientos de datos, por ejemplo, para mantener una lista ordenada como en la figura:



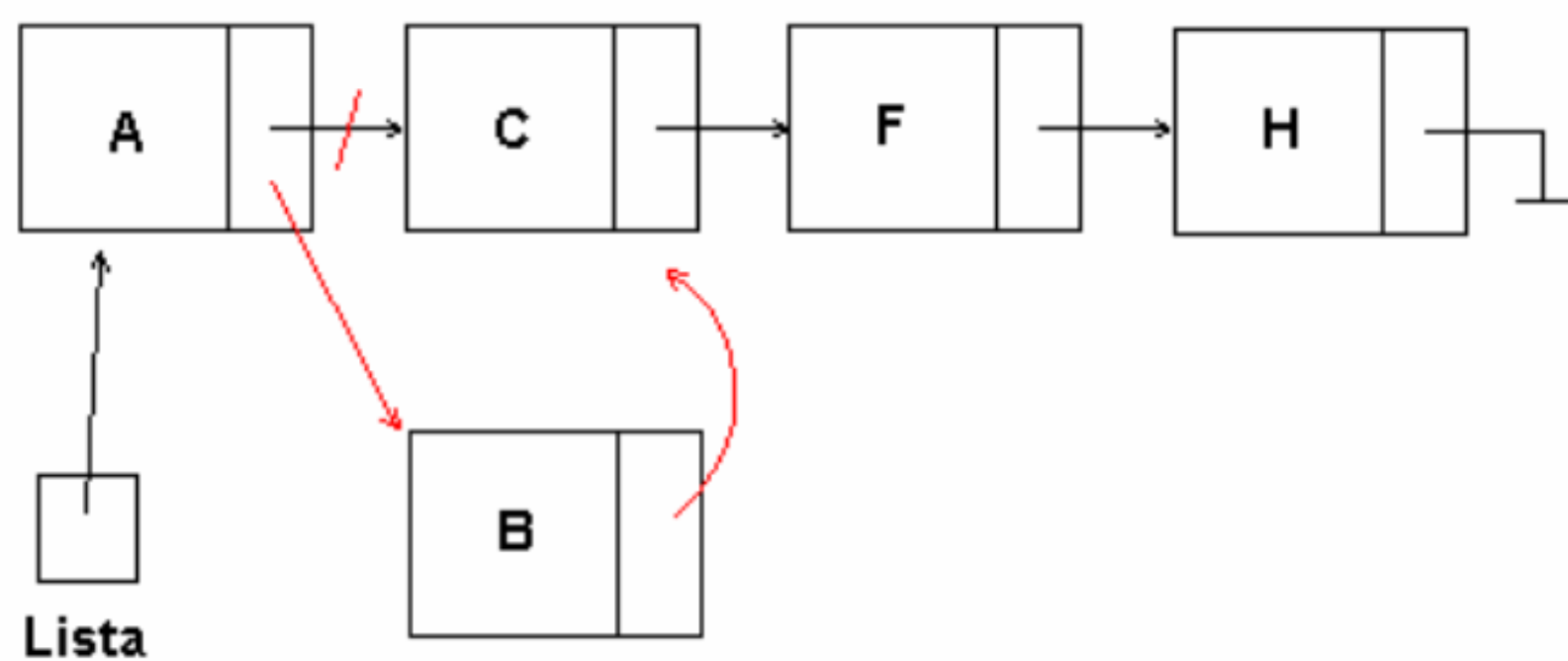
Una variante más compleja es la que llamamos lista con cursores, en donde en cada posición se almacena el dato en sí y un índice que corresponde a la siguiente posición, de esta forma el último dato que se ingresa va siempre a la posición final, y actualizamos solamente un índice sin hacer ningún corrimiento.



Lista: implementación dinámica

Es la implementación más utilizada, guardamos solo un puntero al primer nodo. Los nodos se crean en forma dinámica. En el alta solo hay que ajustar dos punteros.

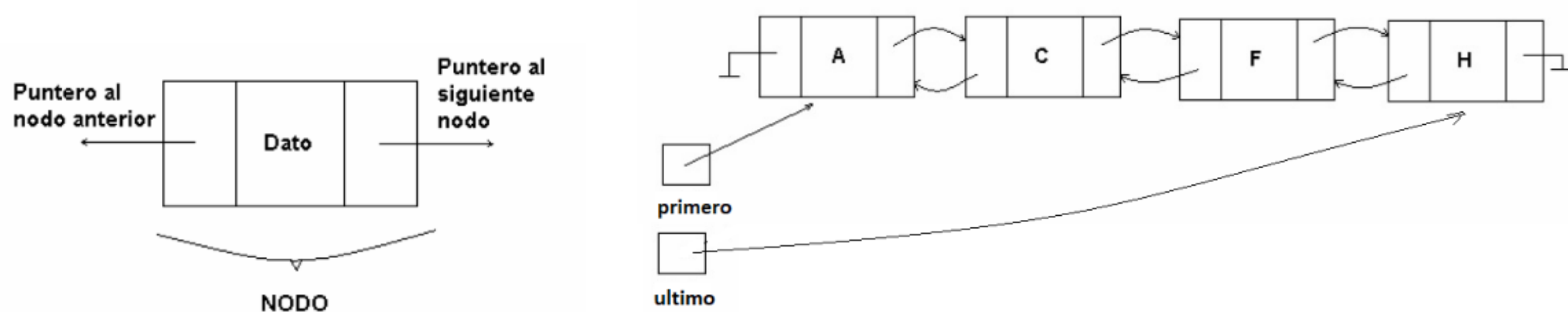
Siguiendo el ejemplo anterior:



Listas doblemente enlazadas

En las listas simplemente enlazadas no podemos acceder a un nodo anterior sin recorrer toda la lista desde el principio.

Las doblemente enlazadas agregan a la clase Nodo un puntero más: puntero al nodo anterior. De esta forma podemos recorrerla en ambos sentidos.



Hay que tener en cuenta que, al insertar un nuevo nodo, se deben ajustar cuatro punteros: los dos del nuevo nodo, el anterior del siguiente y el posterior del anterior.

Una implementación de este tipo tendría sentido en aplicaciones donde se debe estar accediendo frecuentemente tanto a elementos anteriores de cierto nodo como posteriores, por ejemplo los algoritmos de comprensión o suavizado de curvas, etc.

También tiene sentido agregar un puntero más a la clase que sería el actual. Este puntero señalará en cada momento al nodo donde se está poniendo el foco.

Listas circulares

Una lista circular es una lista en donde el último nodo apunta al Primero.

Esta implementación es útil para el tratamiento de datos que se reciben en forma continua, como por ejemplo datos que devuelve un sensor, que pueden ser temperaturas, flujos, etc.

Como los procesos son continuos en el tiempo, una lista circular de n nodos nos permite tener vigentes los últimos n datos.

Las listas circulares podrían tener a lo sumo n datos.

De esta forma, al agregar un dato se pondría en el lugar del Primer dato (el más viejo se elimina) y la referencia al primero pasaría a la siguiente posición.

